[승코의 차근차근 Flutter] 01. 다트 기본 문법

✅ 1. Dart 기초 문법 정리

📌 1. `main()` 함수 (프로그램의 시작점)

```
void main() {
  // 여기에 코드 작성
}
```

- `main()`은 Dart 프로그램이 실행될 때 **가장 먼저 호출되는 함수**예요.
- `void`는 **리턴값이 없다**는 의미.
- 괄호 안이 비어 있으면 매개변수를 받지 않음.

📌 2. 주석

```
// 한 줄 주석

/*
여러 줄 주석
*/

/**
* 문서 주석 (///로도 가능)
* DartDoc에서 문서화용으로 사용
*/
```

📌 3. `print()` 함수

```
print('Hello World');
```

- 콘솔에 메시지를 출력합니다.

📌 4. 변수 선언

▶️ var

```
var name = '코드팩토리';
```

- **타입 자동 추론**. 문자열이 들어가면 `String`으로 고정됨.
- 한 번 타입이 정해지면 **다른 타입은 넣을 수 없음**.

▶️ dynamic

```
dynamic name = '코드팩토리';
name = 123; // 가능
```

- 타입이 **고정되지 않음**. 모든 타입을 저장할 수 있음.
- 하지만 **타입 안정성이 떨어져** 가능하면 피하는 게 좋아요.

▶️ final vs const

구분	설명
`final`	값을 **한 번만 할당 가능**, 런타임에 결정됨
`const`	**컴파일 시점에 고정**, 빌드 타임 상수

```
final timeNow = DateTime.now(); // 가능
const timeNow = DateTime.now(); // ❌ 에러
```

📌 5. 타입 명시

```
String name = '홍길동';
int age = 30;
double pi = 3.14;
bool isTrue = true;
```

- 명시하면 코드 가독성이 좋아져 유지보수에 유리.

✅ 2. 컬렉션

📌 2.1 List (배열)

```
List<String> members = ['지수', '제니', '리사'];
print(members[0]); // 인덱스로 접근
```

주요 메서드

- `add()` : 요소 추가
- `where()` : 조건 필터링
- `map()` : 요소 변형
- `reduce()` : 하나로 모으기
- `fold()` : reduce와 비슷하되 초기값 설정 가능

📌 2.2 Set (중복 없는 리스트)

```
Set<String> names = {'지수', '제니', '지수'};
print(names); // 중복 제거됨
```

- `contains()` : 포함 여부 확인
- `.toList()` , `Set.from(List)` : 형 변환 가능

📌 2.3 Map (key-value 쌍)

```
Map<String, String> dictionary = {
  'Harry': '해리',
  'Ron': '론',
};

print(dictionary['Harry']);
```

- `dictionary.keys`, `dictionary.values`로 키/값 가져오기
- `Map<키 타입, 값 타입> 이름 = { }` 형태

📌 2.4 enum (선택지 제한)

```
enum Status { approved, pending, rejected }

Status status = Status.approved;
```

- `Status.values`로 전체 enum 값 조회 가능
- 오류 방지 및 자동완성에 유리

✅ 3. 연산자

📌 3.1 수치 연산자

```
int a = 5;
a += 2; // 7
a *= 3; // 21
```

- `+`, , , `/`, `%`, `++`, `-` 등

📌 3.2 null 관련 연산자

```
String? name = null;

name ??= '기본값'; // null이면 대입
```

- `?`: null 허용
- `??`: null일 때만 처리
- `!`: null 아님을 강제

📌 3.3 타입 비교

```
print(123 is int); // true
print('hi' is! int); // true
```

✅ 4. 제어문

📌 4.1 if / else if / else

```
if (score >= 90) {
  print('A');
} else if (score >= 80) {
  print('B');
} else {
  print('F');
}
```

📌 4.2 switch-case

```
switch (status) {
  case Status.approved:
    print('승인됨');
    break;
  case Status.pending:
    print('대기 중');
    break;
  default:
    print('알 수 없음');
}
```

📌 4.3 for / for-in

```
for (int i = 0; i < 3; i++) {
  print(i);
}

for (var name in names) {
  print(name);
}
```

📌 4.4 while / do-while

```
int i = 0;
while (i < 3) {
  print(i);
  i++;
}
```

✅ 5. 함수

📌 5.1 기본 함수

```
int add(int a, int b) {
  return a + b;
}
```

📌 5.2 네임드 파라미터

```
int add({required int a, int b = 0}) {
  return a + b;
}

add(a: 1, b: 2); // 순서 상관 없음
```

📌 5.3 익명 함수 / 람다 함수

```
numbers.map((n) => n * 2).toList();
```

- `=>`는 간단한 함수 정의를 위해 사용
- 코드 블록이 1줄일 때만 사용 가능

📌 5.4 typedef (함수 타입 저장)

```
typedef Operation = void Function(int a, int b);

void add(int a, int b) => print(a + b);

Operation op = add;
op(1, 2); // 실행됨
```

✅ 6. 예외 처리 (try-catch-finally)

📌 기본 구조

```
try {
  // 예외 발생 가능 코드
} catch (e) {
  print('에러 발생: $e');
} finally {
  print('무조건 실행됨');
}
```

- **try**: 문제가 생길 수 있는 코드
- **catch**: 예외 발생 시 실행
- **finally**: 에러 발생 여부와 관계 없이 항상 실행됨

📌 강제로 에러 발생

```
throw Exception('에러 발생!');
```

📌 catch의 추가 정보

```
catch (e, stackTrace) {
  print(e);
  print(stackTrace);
}
```

- 에러 로그 디버깅에 유용

🎁 Dart 문법 전체 요약 키워드

주제	핵심 키워드
변수	`var`, `final`, `const`, `dynamic`
타입	`String`, `int`, `double`, `bool`, `List`, `Set`, `Map`, `enum`
제어문	`if`, `else`, `switch`, `for`, `while`, `do while`
함수	기본, 네임드, 선택, 람다, typedef
예외 처리	`try`, `catch`, `finally`, `throw`, `on`, `rethrow`
컬렉션 함수	`add`, `where`, `map`, `reduce`, `fold`
null 처리	`?`, `!`, `??`, `??=`, `is`, `is!`